E3.4

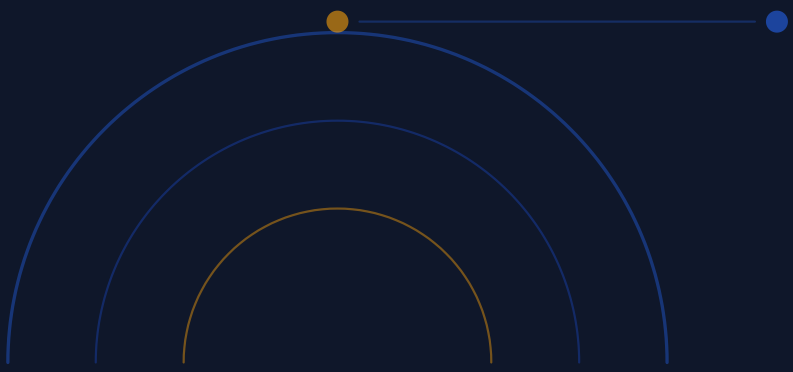
Proyecto final y memoria

La obra y su proceso

Landing page

Memoria

Defensa oral



UNIVERSIDAD PEDAGÓGICA Y TECNOLÓGICA DE COLOMBIA

Tecnología de Programación de Sistemas Informaticos

Unidad 3 - DoK 4

Guion literal de la presentación — 10 minutos

Archivo para proyectar: `presentacion_resumia.pptx` (14 diapositivas, 16:9).

Cómo usarlo: abre el PPTX en PowerPoint y entra en **Presentación → Vista Moderador**. Las notas de cada diapositiva contienen exactamente el texto de este documento.

Duración objetivo: 10 minutos 10 segundos, con margen para preguntas.

Video de la presentación: https://youtu.be/RWPANQKzXgY

Diapositiva 1 — Portada (0:00–0:30)

Di exactamente:

> Buenos días. Mi nombre es Juan Pablo Barrero y hoy les voy a mostrar ResumIA. Es una landing page para una beta que resume PDFs con inteligencia artificial. Pero la entrega de la Unidad 3 va más allá de la página: también incluye la persistencia del formulario, una auditoría del código que generó la IA, un sistema de componentes y la memoria del proceso. La idea es que en diez minutos vean no solo qué quedó, sino cómo lo construí y cómo comprobé que funciona. Arranco por el problema.

Diapositiva 2 — El problema (0:30–1:15)

Di exactamente:

> ResumIA nace de algo que nos pasa a todos los que estudiamos o investigamos. Un estudiante de posgrado, un profesor o un investigador puede tener diez, quince o veinte PDFs por semana. Y el problema no es que falte información: el problema es el tiempo que se va en decidir cuáles vale la pena leer. Antes de leer en profundidad, uno tiene que filtrar. Y ese filtro consume energía. ResumIA no reemplaza la lectura académica: ataca esa primera capa, la de organizar y decidir qué documento merece la pena. Esa es la idea que está detrás de toda la página.

Diapositiva 3 — La propuesta de valor (1:15–2:00)

Di exactamente:

> Y la propuesta se resume en tres cosas. Primero, tiempo: convertir cincuenta páginas en ideas clave, conclusiones y próximos pasos. Segundo, rigor: no se trata de resumir por resumir, sino de conservar los argumentos y los matices, porque el criterio sigue siendo del usuario. Y tercero, privacidad: los documentos se procesan de forma cifrada y no se comparten con terceros. La página tiene un solo objetivo: que la persona deje su correo para entrar a la beta. Por eso el recorrido va de la promesa a los beneficios, luego a los testimonios, resuelve dudas con el FAQ, mete urgencia con el contador y termina en el formulario. Y el diseño acompaña: el azul da confianza, el naranja marca la acción, Lora pone el tono editorial e Inter mantiene todo legible.

Diapositiva 4 — Arquitectura de la landing (2:00–2:50)

Di exactamente:

> En cuanto a la arquitectura, quise que fuera simple a propósito. HTML5 semántico, CSS mobile-first con 32 variables y JavaScript puro, sin frameworks. El comportamiento está separado en cinco módulos que se cargan al final del body, cada uno dentro de una IIFE: core, persistencia, FAQ, contador y progreso de lectura. Eso evita variables globales y hace que cada archivo se pueda explicar en una frase. El sitio vive en GitHub Pages, que solo sirve archivos estáticos. Por eso el formulario no tiene un backend propio: el envío va a un servicio externo, Formspree en producción. Para la demo uso httpbin, que me permite ver el POST y el 200 sin depender de una cuenta.

Diapositiva 5 — Persistencia: seis requerimientos (2:50–3:40)

Di exactamente:

> Para la persistencia definí seis datos. El nombre, para no obligar a reescribirlo si la persona vuelve. El correo, que es el lead y no se puede perder. El perfil, para segmentar la beta. El consentimiento, para conservar la decisión entre recargas y no confundir "no aceptó" con "nunca se le preguntó". El estado de conversión, para no volver a mostrar el formulario a quien ya se registró. Y la fecha de la cohorte, para que la urgencia no se reinicie en cada visita. Todos usan el prefijo resumia_. Elegí localStorage y no cookies ni sessionStorage porque sobrevive al cierre, no se envía en cada petición y no expone datos personales en la red.

Diapositiva 6 — Persistencia: el flujo completo (3:40–4:25)

Di exactamente:

> El flujo es sencillo. Mientras la persona escribe, el evento input guarda cada campo. Si recarga, el borrador se restaura solo. Cuando envía, se valida, se hace el POST con fetch y AbortController, se comprueba response.ok, se guarda la marca de usuario convertido, se limpian las claves del borrador y aparece el agradecimiento. La próxima vez que entra, el formulario ya no está. Y eso es importante: después de la conversión no dejamos datos personales guardados en el navegador.

Diapositiva 7 — Auditoría: catorce hallazgos (4:25–5:10)

Di exactamente:

> La auditoría fue la parte más incómoda, pero también la que más me enseñó. Generé la sección en una sesión limpia de AI Studio y la revisé antes de ejecutarla. Encontré catorce hallazgos; cuatro eran críticos. El saludo usaba innerHTML con el nombre del usuario, o sea, un XSS. El envío usaba un XHR síncrono que congelaba la interfaz. document.write reemplazaba el documento y el reset posterior lanzaba un TypeError. Y un console.log imprimía el correo y el consentimiento. La consola de DevTools lo confirma: la fuga de datos, el error en la línea 83 y la advertencia de formulario desconectado. No fue una revisión teórica: se ve en la pantalla.

Diapositiva 8 — Auditoría: las correcciones (5:10–5:55)

Di exactamente:

> Después vino la corrección. Apliqué el ciclo AVIA: analizar, verificar, iterar y ajustar. En analizar inventarié el código antes de ejecutarlo; en verificar usé la consola real; en iterar corregí con prompts y cambios manuales; y en ajustar reescribí el prompt base con restricciones de seguridad. Los tres cambios clave fueron: innerHTML por textContent, el XHR síncrono por fetch asíncrono con AbortController, y document.write por un bloque de éxito que ya existe en el DOM. Además definí un protocolo de doce ítems: cuatro de seguridad, cuatro de calidad y cuatro de rendimiento.

Diapositiva 9 — Sistema de componentes (5:55–6:40)

Di exactamente:

> Para que la página no creciera sin orden, definí un sistema de componentes. Son dieciséis tokens documentados: color, tipografía, espaciado, radios, sombras. Y una convención: resumia-componente__elemento–estado. Cada snippet consume al menos tres variables, vive en su propia IIFE y se integra con una clase contenedora única. El modo oscuro sale gratis: se redefinen los tokens y los componentes se adaptan sin reglas nuevas.

Diapositiva 10 — Los cuatro snippets (6:40–7:25)

Di exactamente:

> Los cuatro snippets son el hero, el acordeón de preguntas, el contador de la cohorte y la barra de progreso con el botón de volver arriba. Cada uno tiene cinco tokens, su IIFE y su clase. El protocolo exige script al final del body y ninguna variable global. La idea es que el sistema no sea una colección de archivos, sino un contrato: cualquier componente nuevo se puede evaluar con las mismas reglas.

Diapositiva 11 — Resultados verificables (7:25–8:10)

Di exactamente:

> Los resultados son medibles. Lighthouse, en móvil, da 100 en accesibilidad, buenas prácticas y SEO, y entre 97 y 100 en rendimiento. La consola no tiene errores ni respuestas 4xx, y el validador W3C no reporta mensajes. La página responde en 375, 768 y 1280 píxeles. Y todo está publicado en GitHub Pages, con el repositorio y los PDFs enlazados desde el README.

Diapositiva 12 — Evidencia de la persistencia (8:10–8:55)

Di exactamente:

> Estas son las dos capturas que respaldan la persistencia. A la izquierda, el panel Application muestra las claves resumia_form_ con el borrador. A la derecha, Network registra el POST con estado 200 y la respuesta del servicio. Después del envío, las claves del borrador desaparecen y quedan solo la marca de

> usuario convertido y el nombre. Esa es la prueba de que la limpieza ocurre de verdad y no solo está en el código.

Diapositiva 13 — Lo que aprendí (8:55–9:40)

Di exactamente:

> ¿Y qué me llevo de todo esto? Que la IA no reemplaza al desarrollador: desplaza el valor del trabajo. Cuando el código se genera en segundos, lo escaso es decidir qué construir, verificarlo y responder por él. Las competencias que no se delegan son el criterio, la auditoría y la evidencia. Gemini fue muy bueno para la estructura y el copy, y más débil en seguridad y verificación. Por eso el flujo que voy a repetir tiene dos condiciones: revisión humana en todo lo que toque datos personales y un protocolo escrito antes de generar, no después.

Diapositiva 14 — Cierre (9:40–10:10)

Di exactamente:

> ResumIA está publicado en kanonufo.github.io/landing-page-ia. En el repositorio están el código, los entregables y las evidencias, y el tablero Kanban está vinculado. Con eso cierro. Muchas gracias, y quedo atento a sus preguntas.

Antes de presentar

1. Abre el sitio publicado y comprueba que carga: https://kanonufo.github.io/landing-page-ia/.
2. Ten a mano la demo por si falla internet: `docs/unidad3/E32_persistencia/demo_persistencia.mp4`.
3. Abre el PPTX en **Presentación → Vista Moderador** para ver las notas mientras proyectas.
4. Si el tiempo se acorta, omite la diapositiva 10 (snippets) y conserva 5, 7 y 11.
5. Si preguntan por el backend: no hay backend propio; Formspree recibe el POST y GitHub Pages sirve el sitio estático.

Preguntas previsibles y respuestas breves

- **¿Por qué httpbin y no Formspree en la demo?** Porque permite verificar el POST y el 200 sin una cuenta; el cambio es una línea en `data-endpoint`.
- **¿Qué pasa en modo privado?** El formulario funciona; solo se pierde la persistencia entre recargas.
- **¿Cómo se evita el XSS?** `textContent`, validación y prohibición de `innerHTML` con datos de usuario.
- **¿Por qué no cookies?** Se envían en cada petición y añaden exposición de datos personales innecesaria.
- **¿Qué fue lo más difícil?** La auditoría: distinguir entre “funciona” y “es correcto”.